

TPL Async/Await

Async/Await

- Ermöglicht bessere Kontrolle über Vordergrund/Hintergrund Aufgaben
- async und await Keywords
- await kann nur in einer Methode die als async markiert ist verwendet werden
- async Methoden können nur bestimmte Typen (u.a. void, Task, Task<T>, ValueTask<T>) zurückgeben

```
public async void FetchData()  
{  
    using HttpClient client = new HttpClient();  
    await client.GetAsync("URL");  
}
```



(awaitable) Task<HttpResponseMessage> HttpClient.GetAsync(string? requestUri)
Send a GET request to the specified Uri as an asynchronous operation.

Returns:

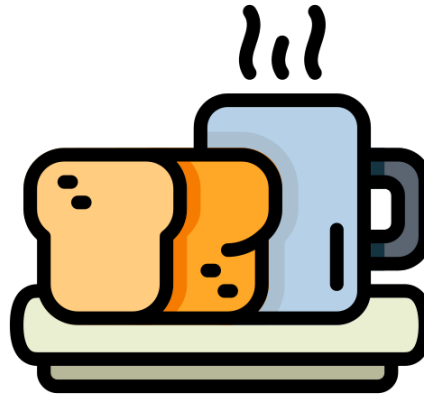
The task object representing the asynchronous operation.

Task.Wait() vs. async/await

- Task.Wait() / Task.Result
 - legt die Aufgabe beim Aufruf von Wait() auf den Main Thread
 - Main Thread wird geblockt bis der Task fertig ist
 - Problematisch in vielen Frameworks (WinForms, WPF, MAUI, ASP.NET, Blazor, ...)
- await
 - lässt die Aufgabe im Hintergrund weiterlaufen / parallel zum Main Thread
 - Erzeugt dafür eine State Machine
 - Ergebnis wird zurückgegeben wenn der Task fertig ist
 - Main Thread läuft ununterbrochen weiter

Beispiel

- Frühstück zubereiten
 - Toast toasten → 2 Minuten
 - Teller/Besteck + Tasse herrichten → 30 Sekunden
 - Kaffee zubereiten (basiert auf Tasse herrichten) → 30 Sekunden
- Programm Synchron: 3 Minuten
- Realität: Toast in den Toaster → Tasse unter Kaffeemaschine → Essen servieren (2 Minuten)



Checkliste

- Aufgabe Starten und in Variable speichern

```
using HttpClient client = new();  
Task<HttpStatusCode> request = client.GetAsync(url); //Starte den Request
```

- Zwischenschritte (Benachrichtigung an den User, ...)

- Optional

```
ReqButton.IsEnabled = false; //Zwischenschritt  
Output.Text = "Text wird geladen..."; //Zwischenschritt
```

- Auf Ergebnis warten

```
HttpStatusCode response = await request; //Warte auf das Ende des Requests
```



Unterschiede zw. Rückgabewerten

- Verschiedene Aufbauten haben verschiedene Effekte
- `async void`
 - Kann in sich selbst `await` verwenden
 - Kann selbst **nicht** `awaited` werden
 - Wird verwendet, um lange „await-Ketten“ zu beenden
- `async Task`
 - Kann in sich selbst `await` verwenden **und** selbst `awaited` werden
 - Hat keinen Rückgabewert
- `async Task<T>`
 - Die gleichen Eigenschaften wie `async Task`, muss aber einen Wert zurückgeben



Wichtige Notizen

- Wenn eine Methode, die einen Task zurückgibt, ausgeführt wird, wird diese als Task ausgeführt
 - Diese Methode läuft dann parallel zum Main Thread
 - Kein `t.Start()` oder `Task.Run(...)` erforderlich
- Die Task-Klasse selbst kann awaited werden
 - Damit können die meisten langandauernden Operationen parallelisiert werden
- Bei Rückgabewerten ohne Generic (`<T>`) ist kein return innerhalb der Methode notwendig

Mehrere Tasks verwalten

- Gestartete Tasks können in einer Liste gespeichert werden
 - Alternative: Mehrere Tasks über eine Schleife starten
- Mithilfe von **await Task.WhenAll(...)** können mehrere Tasks gemeinsam awaited werden
- Im Debugger kann der Status von mehreren Tasks inspiziert werden



Asynchrone Datenquelle

- Hierfür wird das Interface `IAsyncEnumerable` verwendet
- Diese nimmt ein Generisches Typargument, um den Rückgabewert zu definieren
- Mithilfe von `await foreach` kann diese Datenquelle dann angesprochen werden
- Werte werden herausgegeben, sobald diese verfügbar sind

```
public static async IAsyncEnumerable<int> Generiere()  
{  
    for (int i = 0; i < 20; i++)  
    {  
        await Task.Delay(100);  
        yield return i;  
    }  
}  
  
public static async void GebeZahlenAus()  
{  
    await foreach (var zahl in Generiere())  
    {  
        Console.WriteLine(zahl);  
    }  
}
```